Made by Chaima EL HADAD, Mariam ABDILLAHI AHMED, Kelly HILLAH

Data Visualisation - Project I: NHGIS and the US county population

PYTHON :

1.Preliminaries

In this section, you will see the ETL (Extract, Transform, Load) pipeline that was created to process US Census spatial data using the IPUMS NHGIS API. It includes geographic cleaning, the creation of a reproducible workflow to programmatically download 2010 county shapefiles from IPUMS NHGIS, and the handling of complex nested ZIP archives using temporary directories. It also filters the data to the Continental United States (excludes Alaska, Hawaii, and territories), and includes final structural verification to ensure the data is fit for a choropleth visualization.

```python
import os
import glob
import zipfile
import tempfile
import geopandas as gpd
from ipumspy import IpumsApiClient, AggregateDataExtract

# Load the IPUMS API key from a local text file
with open('ipums_key.txt', 'r') as file:
    IPUMS_API_KEY = file.read().strip()

# Initialize the IPUMS API client
ipums = IpumsApiClient(IPUMS_API_KEY)

# Define the data extract request for 2010 County shapefiles from NHGIS
nhgis_ext = AggregateDataExtract(
    collection="nhgis",
    description="County shapefiles (2010)",
    shapefiles=["us_county_2010_tl2010"]
)

# Submit the extract request to IPUMS servers
nhgis_ext_submitted = ipums.submit_extract(nhgis_ext)
```

```python
# Pause execution until the server finishes processing the extract
ipums.wait_for_extract(nhgis_ext_submitted)

# Set the download destination to the current directory
download_dir = os.path.expanduser(".")
ipums.download_extract(nhgis_ext_submitted, download_dir=download_dir)

# Identify the downloaded zip file by looking for 'nhgis' and 'shape' in the filename
# We select the most recently modified file in case multiple extracts exist
shape_zips = glob.glob(os.path.join(download_dir, "nhgis*shape*.zip"))
shape_zip = max(shape_zips, key=os.path.getmtime)
print(f"Using: {shape_zip}")

# Define a function to extract and read the .shp file from zip archives
def read_shp_from_zip(shape_zip):
    # Use a temporary directory to handle file extraction cleanly
    with tempfile.TemporaryDirectory() as tmpdir:
        # Extract the main zip file
        with zipfile.ZipFile(shape_zip) as z:
            z.extractall(tmpdir)
        # Check for and extract any zip files (NHGIS often nests archives)
        for inner_zip in glob.glob(os.path.join(tmpdir, "**", "*.zip"), recursive=True):
            with zipfile.ZipFile(inner_zip) as z:
                z.extractall(tmpdir)
        # Locate the .shp file within the temporary directory
```

```python
        # Locate the .shp file within the temporary directory
        shp_files = glob.glob(os.path.join(tmpdir, "**", "*.shp"), recursive=True)
        if not shp_files:
            raise FileNotFoundError("Aucun .shp trouvé")
            # Load the shapefile into a GeoDataFrame
        return gpd.read_file(shp_files[0])

# Execute the extraction function
shape_us = read_shp_from_zip(shape_zip)

# Convert State FIPS codes to integers for easier filtering
# Filter for Continental US only (States < 57, excluding Alaska (02) and Hawaii (15))
shape_us = (
    shape_us
    .assign(STATEFP10=lambda df: df["STATEFP10"].astype(int))
    .query("STATEFP10 < 57 and STATEFP10 not in [2, 15]")
)

# Display the final data dimensions and the first few rows
print(shape_us.shape)
shape_us.head()
```

Output :

```
Using: ./nhgis0126_shape.zip
(3109, 21)
```

	STATEFP10	COUNTYFP10	COUNTYNS10	GEOID10	NAME10	NAMELSAD10	LSAD10	CLASSFP10	MTFCC10	CSAFP10	...	METDIVFP10	FUNCSTAT10	ALAND10	AWATER10
2	28	107	00695776	28107	Panola	Panola County	06	H1	G4020	None	...	None	A	1774515519	51767046
3	28	101	00695773	28101	Newton	Newton County	06	H1	G4020	None	...	None	A	1497282694	3879399
4	28	027	00695738	28027	Coahoma	Coahoma County	06	H1	G4020	None	...	None	A	1430818231	79539470
5	22	065	00558088	22065	Madison	Madison Parish	15	H1	G4020	None	...	None	A	1617283455	68538668
6	51	540	01789068	51540	Charlottesville	Charlottesville city	25	C7	G4020	None	...	None	F	26517362	52974

```python
from ipumspy import TimeSeriesTable

# Define a new NHGIS data extract request
# that targets the 'CW8' time series table for the year 2010 at the county level
nhgis_data_ext = AggregateDataExtract(
    collection="nhgis",
    description="CW8 county data 2010",
    time_series_tables=[
        TimeSeriesTable(
            name="CW8",
            geog_levels=["county"],
            years=["2010"]
        )
    ])

# Submitting and waiting the server for the data extract request to the IPUMS API
data_extract = ipums.submit_extract(nhgis_data_ext)
ipums.wait_for_extract(data_extract)

# Download the completed extract to the previously defined download directory
ipums.download_extract(data_extract, download_dir=download_dir)
```

```python
import pandas as pd
import zipfile
import glob
import os

# Locate the most recently downloaded NHGIS ZIP file in the specified directory
# glob.glob finds all files matching the pattern nhgis*csv.zip
data_zip = max(glob.glob(os.path.join(download_dir, "nhgis*csv.zip")), key=os.path.getmtime)

# Open the ZIP archive in reading mode
with zipfile.ZipFile(data_zip, 'r') as z:
    # Identify the main data file (ending in .csv) within the archive
    csv_name = [f for f in z.namelist() if f.endswith('.csv')][0]
    # Open the CSV file and load it into a pandas DataFrame
    with z.open(csv_name) as f:
        df_counts = pd.read_csv(f)

# Initialize a new DataFrame to store processed geographic and demographic data
df_wrangled = pd.DataFrame()
df_wrangled['GISJOIN'] = df_counts['GISJOIN']
df_wrangled['STATE']   = df_counts['STATE']

# # Aggregate White population (combining specific NHGIS code columns : AA + AG)
df_wrangled['pop_white'] = (
    df_counts['CW8AA2010'] + df_counts['CW8AG2010']
```

```python
# Aggregate Black/African American population (AB + AH)
df_wrangled['pop_black'] = df_counts['CW8AB2010'] + df_counts['CW8AH2010']

# Aggregate American Indian and Alaska Native population (AC + AI)
df_wrangled['pop_native'] = df_counts['CW8AC2010'] + df_counts['CW8AI2010']

# Aggregate Asian population (AD + AJ)
df_wrangled['pop_asian'] = df_counts['CW8AD2010'] + df_counts['CW8AJ2010']

# Aggregate Other categories
df_wrangled['pop_other'] = (
    df_counts['CW8AE2010'] + df_counts['CW8AK2010']
)

# Calculate total population by summing all aggregated race columns horizontally
df_wrangled['total_pop'] = df_wrangled[['pop_white', 'pop_black', 'pop_native', 'pop_asian', 'pop_other']].sum(axis=1)

# Merging the demographic data with the shapefile (GeoDataFrame) using the GISJOIN key
# Inner join ensures only matching records are kept in the final spatial dataset
shape_final = shape_us.merge(df_wrangled, on="GISJOIN", how="inner")

print(shape_final[['NAME10', 'pop_white', 'total_pop']].head())
```

Output :

```
            NAME10  pop_white  total_pop
0           Panola      17161      34402
1           Newton      13734      21531
2          Coahoma       5989      26019
3          Madison       4498      11982
4  Charlottesville      30031      42166
```

```python
shape_final
```

	STATEFP10	COUNTYFP10	COUNTYNS10	GEOID10	NAME10	NAMELSAD10	LSAD10	CLASSFP10	MTFCC10	CSAFP10	...	Shape_area	Shape_len	geometry
0	28	107	00695776	28107	Panola	Panola County	06	H1	G4020	None	...	1.826283e+09	174069.544677	POLYGON ((573901.44 -351322.648 573921.329 -.
1	28	101	00695773	28101	Newton	Newton County	06	H1	G4020	None	...	1.501162e+09	155201.497134	POLYGON ((663920.71 -565257.263 663677.888 -.
2	28	027	00695738	28027	Coahoma	Coahoma County	06	H1	G4020	None	...	1.510358e+09	253786.233020	POLYGON ((510444.05 -329388.382 510631.476 -.
3	22	065	00558088	22065	Madison	Madison Parish	15	H1	G4020	None	...	1.685822e+09	307961.992036	MULTIPOLYGON (((464244.96 -583026.069 464871.
4	51	540	01789068	51540	Charlottesville	Charlottesville	25	C7	G4020	None	...	2.657033e+07	37764.013438	POLYGON ((1514588.51

2.Task 1: County population barplots

This exercise validates the 2010 United States Census data for Los Angeles County (FIPS 06037) by comparing the data in the dataset to the reported numbers in the Wikipedia entry, and calculates the raw population counts and the percentage share of the population for each racial category (White, Black or African American, American Indian and Alaska Native, Asian, and some other race) for Los Angeles County. The side-by-side plot with direct labeling (text annotations right on the bars) shows that the data is consistent with the Wikipedia entry (50.3% White, 13.7% Asian), and the raw data is presented in a way that is easy to verify before further analysis by using the same color palette throughout.

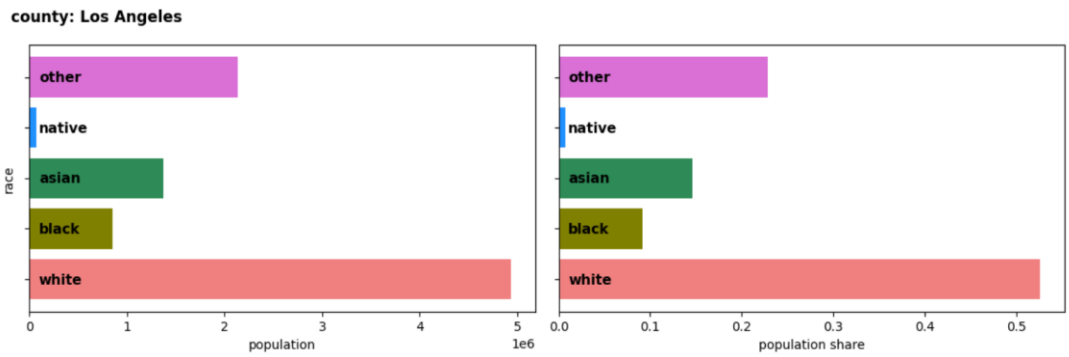
```python
!pip install "numpy<2"
```

```
Requirement already satisfied: numpy<2 in /usr/local/lib/python3.12/dist-packages (1.26.4)
```

```python
import numpy as np
import matplotlib.pyplot as plt

# Extract the specific row for Los Angeles and select the first occurrence
# .iloc[0] converts the single row DataFrame into a Series
la = shape_final[shape_final['NAME10'] == 'Los Angeles'].iloc[0]

# Here we define the categories (races) and retrieve their corresponding values from the Series
races  = ['other', 'native', 'asian', 'black', 'white']
pops   = [la['pop_other'], la['pop_native'], la['pop_asian'],
          la['pop_black'], la['pop_white']]

# Calculate the sum to determine the share (percentage) of each group
total  = sum(pops)
shares = [p / total for p in pops]

# Create numerical positions for the Y-axis based on the number of categories
y_pos  = np.arange(len(races))
# Custom palette color
colors = ['#DA70D6', '#1E90FF', '#2E8B57', '#808000', '#F08080']

# Initialize a figure with two side-by-side subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
fig.suptitle('county: Los Angeles', x=0.02, ha='left', fontweight='bold')
```

```python
ax1.barh(y_pos, pops, align='center', color=colors)
ax1.set_yticks(y_pos, labels=['''] * len(races))
ax1.set_xlabel('population')
ax1.set_ylabel('race')
ax1.invert_yaxis()

# Add custom labels inside/beside the bars for better readability
for i, (p, race) in enumerate(zip(pops, races)):
    ax1.text(max(pops) * 0.02, i, race, va='center', ha='left',
             fontsize=11, fontweight='bold', color='black')

# Right Subplot is for Population Shares (%)
ax2.barh(y_pos, shares, align='center', color=colors)
ax2.set_yticks(y_pos, labels=['''] * len(races))
ax2.set_xlabel('population share')
ax2.invert_yaxis()

# Adding custom labels for the shares subplot
for i, (s, race) in enumerate(zip(shares, races)):
    ax2.text(max(shares) * 0.02, i, race, va='center', ha='left',
             fontsize=11, fontweight='bold', color='black')

# Adjusting layout to prevent overlapping of labels and subplots
fig.tight_layout()
plt.show()
```

Output:



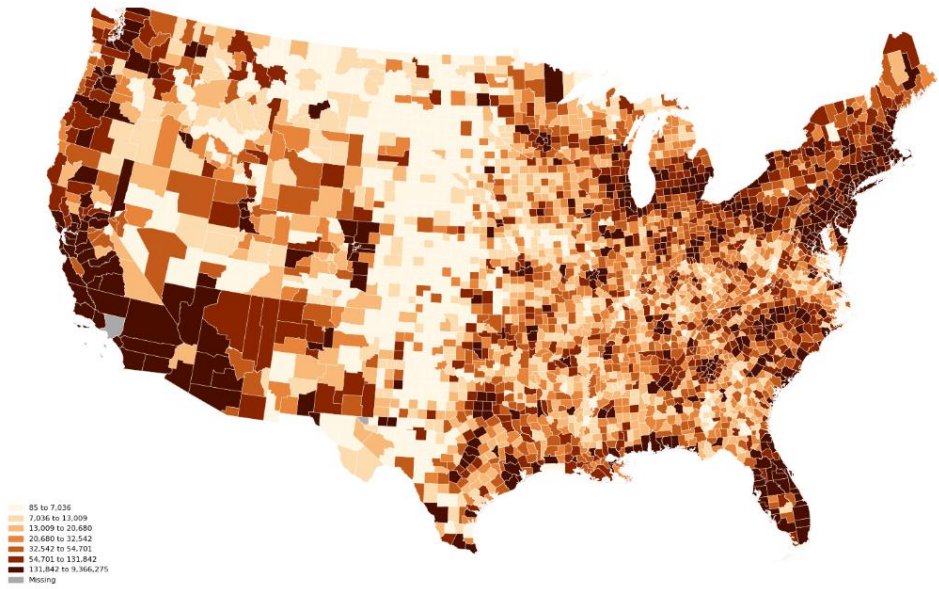
3. Task 2: All counties

In this section, we create a color map of the United States, where colors represent how many people live in each county of the country. The code does this by binning the counties into 7 categories (bins) instead of thousands of different numbers, by picking a light yellow (low population) to dark brown (high population) palette so that crowded counties like New York or Los Angeles can be easily identified, and by including a small legend in the bottom right corner to

explain the color scale. This map essentially takes a long list of numbers and translates it into a visualization of the density of the U.S. population.

```python
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

# Defining the threshold breaks (bins) for the population categories
bins   = [85, 7036, 13009, 20680, 32542, 54701, 131842, 9366275]

# Defining the descriptive labels for each population range
labels = ['85 to 7,036', '7,036 to 13,009', '13,009 to 20,680',
          '20,680 to 32,542', '32,542 to 54,701', '54,701 to 131,842',
          '131,842 to 9,366,275']

# Customing color palette
colors = ['#FFF7E6', '#FDDCB0', '#F9B978', '#E8853A',
          '#C45A1A', '#8B2500', '#4A0E00']

# We use pd.cut to segment the numerical 'total_pop' column into categories
shape_final['pop_cat'] = pd.cut(
    shape_final['total_pop'],
    bins=bins,
    labels=labels,
    include_lowest=True
)

# Mapping the category labels to their corresponding colors
color_map = dict(zip(labels, colors))
shape_final['color'] = shape_final['pop_cat'].astype(str).map(color_map).fillna('#AAAAAA')

fig, ax = plt.subplots(1, 1, figsize=(16, 10))

# Plotting the GeoDataFrame using the color column
shape_final.plot(
    color=shape_final['color'],
    edgecolor='white',
    linewidth=0.3,
    ax=ax
)

ax.set_title('Total Population by County', fontsize=16, fontweight='bold', pad=15)
ax.axis('off')

# Manually creating legend patches based on the color map
patches = [mpatches.Patch(color=c, label=l) for l, c in color_map.items()]
# Adding a patch for areas with no data or values outside the bins
patches.append(mpatches.Patch(color='#AAAAAA', label='Missing'))

# Placing the legend in the bottom left corner
ax.legend(
    handles=patches,
```

```python
ax.legend(
    handles=patches,
    loc='lower left',
    fontsize=8,
    frameon=False,
    title=None
)

plt.tight_layout()
plt.show()
```



4. Task 3: leaflet

In this exercise, we convert the static analysis into an interactive Leaflet application with the Folium library and simplify the geographic shapes to ensure fast performance, convert the shapes to the WGS84 coordinate system, add the ability to highlight the border of a county with blue, add tooltips and popups, and allow a user to click on a county to generate an on-the-fly report for that county including the name and a population breakdown of each race, turning it into a web-based exploratory tool.

```
!pip install folium branca
```

```
Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: branca in /usr/local/lib/python3.12/dist-packages (0.8.2)
Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (1.26.
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinj
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from r
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from r
```

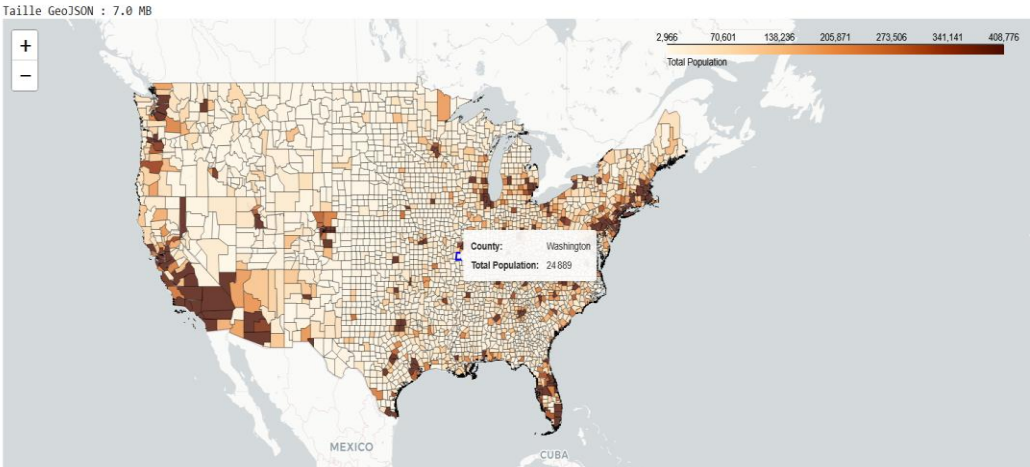
```
import folium
import branca.colormap as cm
import json
import numpy as np
from IPython.display import IFrame

# Converting the coordinate system to WGS84 for compatibility with Folium/Leaflet
shape_wgs84 = shape_final.to_crs(epsg=4326).copy()

# Simplifying geometries to reduce file size and improve map rendering performance
shape_wgs84['geometry'] = shape_wgs84.geometry.simplify(
    # tolerance=0.05 removes small details while preserve_topology maintains shape integrity
    tolerance=0.05,
    preserve_topology=True
)

# Converting the GeoDataFrame to a GeoJSON string to check size, then loading as a dictionary
geojson_str = shape_wgs84.to_json()
print(f"Taille GeoJSON : {len(geojson_str)/1e6:.1f} MB")
geojson_dict = json.loads(geojson_str)
del geojson_str

# Creating a linear colormap based on population quantiles to handle outliers
colormap = cm.LinearColormap(
    colors=['#FFF7E6', '#FDDCB0', '#F9B978', '#E8853A', '#C45A1A', '#8B2500', '#4A0E00'],
```

```python
    vmin=shape_wgs84['total_pop'].quantile(0.05),
    vmax=shape_wgs84['total_pop'].quantile(0.95),
    caption='Total Population'
)

# Centering the map on the Continental US with a light background tile
m = folium.Map(
    location=[39.5, -98.35],
    zoom_start=4,
    tiles='CartoDB positron'
)

# Interactive Layer Configuration
folium.GeoJson(
    geojson_dict,
    name='Counties',
    style_function=lambda feature: {
        'fillColor': colormap(feature['properties']['total_pop'])
                     if feature['properties']['total_pop'] is not None
                     else '#AAAAAA',
        'color':       'black',
        'weight':      0.3,
        'fillOpacity': 0.8,
    },
    highlight_function=lambda feature: {
```

```python
        'fillOpacity': 0.8,
    },
    highlight_function=lambda feature: {
        'fillColor': colormap(feature['properties']['total_pop'])
                     if feature['properties']['total_pop'] is not None
                     else '#AAAAAA',
        'color':       'blue',
        'weight':      2,
        'fillOpacity': 0.9,
    },
    tooltip=folium.GeoJsonTooltip(
        fields=['NAME10', 'total_pop'],
        aliases=['County:', 'Total Population:'],
        localize=True,
        sticky=False
    ),
    popup=folium.GeoJsonPopup(
        fields=['NAME10', 'total_pop', 'pop_white', 'pop_black',
                'pop_asian', 'pop_native', 'pop_other'],
        aliases=['County', 'Total Pop.', 'White', 'Black',
                 'Asian', 'Native', 'Other'],
        localize=True,
        labels=True,
        style="font-family: Arial; font-size: 12px;"
    )
```

```python
# Add the color scale legend to the map
colormap.add_to(m)

# Display the map
m
```

Taille GeoJSON : 7.0 MB



5.Task 4: Shiny

The final task was to develop a dynamic dashboard with the Dash library and callbacks to provide an interactive user interface with interdependent dropdown menus where the selection of a state filters the counties, a map that updates with each click, and that updates the racial population bar charts (from Task 1) whenever a new county is chosen to make this static data and maps into a professional, responsive tool to explore U.S. demographic patterns in real-time.

```
!pip install dash
```

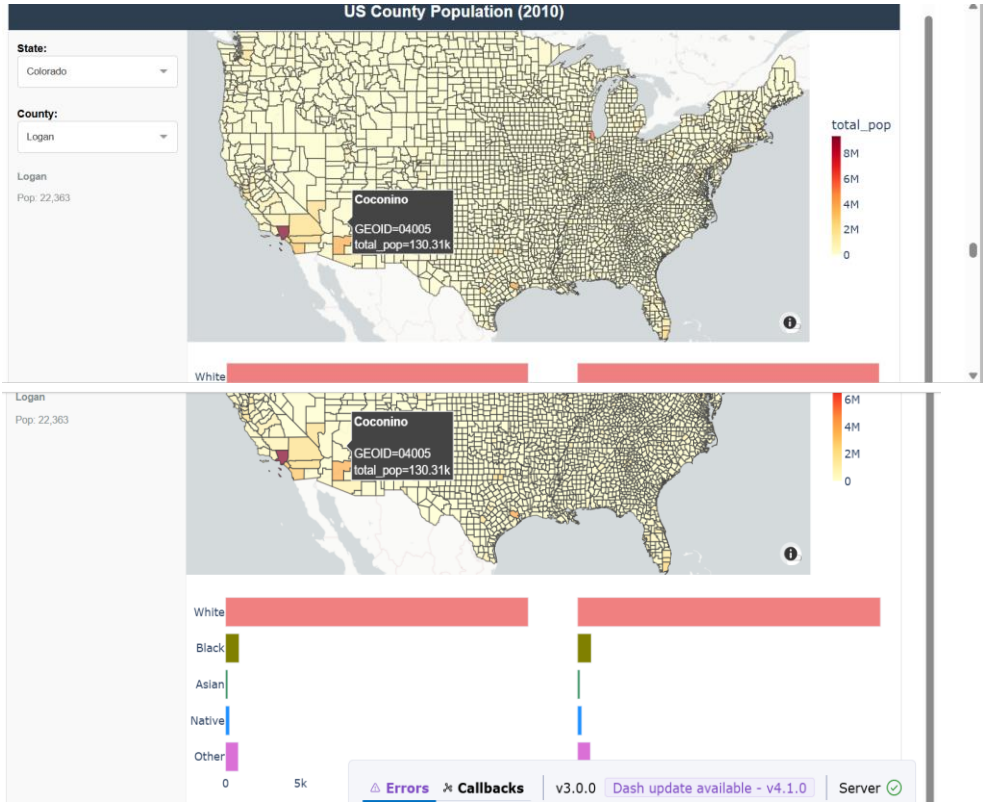
```
Collecting dash
  Downloading dash-4.1.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: Flask<3.2,>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from dash) (3.1.3)
Requirement already satisfied: Werkzeug<3.2 in /usr/local/lib/python3.12/dist-packages (from dash) (3.1.7)
Requirement already satisfied: plotly>=5.0.0 in /usr/local/lib/python3.12/dist-packages (from dash) (5.24.1)
Requirement already satisfied: importlib-metadata in /usr/local/lib/python3.12/dist-packages (from dash) (4.13.0)
Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.12/dist-packages (from dash) (4.15.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from dash) (2.32.4)
Collecting retrying (from dash)
```

```
!pip install pycloudflared -q
from pycloudflared import try_cloudflare
```

```
import dash
from dash import dcc, html, Input, Output, State, ctx, no_update
import plotly.graph_objects as go
import plotly.express as px
import json
import pandas as pd

# Project spatial data to WGS84 (latitude/longitude) for web map compatibility
shape_wgs84 = shape_final.to_crs(epsg=4326).copy()
shape_wgs84 = shape_wgs84.rename(columns={'GEOID10': 'GEOID'})

# Simplifying the geometry to improve web loading speed (lower precision for better performance)
shape_wgs84['geometry'] = shape_wgs84.geometry.simplify(0.1, preserve_topology=True)

# Converting GeoDataFrame to GeoJSON format
geojson_counties = json.loads(shape_wgs84.to_json())

# Define global variables for mapping
NAME_COL = 'NAME10'
states = sorted(shape_wgs84['STATE'].dropna().unique().tolist())
```

```
# Color palette for consistent race categorization across the dashboard
RACE_COLORS = {
    'white': '#F08080', 'black': '#808000',
    'asian': '#2E8B57', 'native': '#1E90FF', 'other': '#DA70D6'
}


app = dash.Dash(__name__)

# Defining the user interface structure
app.layout = html.Div([
    html.H2("US County Population 2010", style={'fontFamily': 'Arial', 'textAlign': 'center'}),
    html.Div([
       # Sidebar: Contains filter controls
        html.Div([
            html.Label("State:", style={'fontWeight': 'bold'}),
            dcc.Dropdown(id='state-dd',
                         options=[{'label': s, 'value': s} for s in states],
                         value='California', clearable=False),
            html.Br(),
            html.Label("County:", style={'fontWeight': 'bold'}),
            dcc.Dropdown(id='county-dd', clearable=False),
        ], style={'width': '250px', 'padding': '20px', 'borderRight': '1px solid #ddd'}),
```

```python
        # Main Area: Contains the map, text info, and charts
        html.Div([
            dcc.Graph(id='map', style={'height': '500px'}),
            html.Div(id='click-info', style={'padding': '10px', 'color': '#666'}),
            dcc.Graph(id='bar-chart', style={'height': '300px'}),
        ], style={'flex': '1'})
    ], style={'display': 'flex'}),
    # Hidden store to trigger the initial map render on page load
    dcc.Store(id='_init', data=True),
])

# callbacks (Interactive Logic)
@app.callback(Output('map', 'figure'), Input('_init', 'data'))
def render_map(_):
    fig = px.choropleth_mapbox(
        shape_wgs84,
        geojson=geojson_counties,
        locations='GEOID',
        featureidkey='properties.GEOID',
        color='total_pop',
        color_continuous_scale='YlOrRd',
        range_color=(shape_wgs84['total_pop'].quantile(0.05),
                     shape_wgs84['total_pop'].quantile(0.95)),
        mapbox_style='carto-positron',
        zoom=3, center={'lat': 39.5, 'lon': -98.35},
        opacity=0.7,
        hover_name=NAME_COL
    )
    fig.update_layout(margin={'r': 0, 't': 0, 'l': 0, 'b': 0})
    return fig

# Syncs the State dropdown when a user clicks on the map
@app.callback(Output('state-dd', 'value'), Input('map', 'clickData'), prevent_initial_call=True)
def update_state_from_map(click_data):
    if not click_data: return no_update
    geoid = click_data['points'][0].get('location', '')
    row = shape_wgs84[shape_wgs84['GEOID'] == geoid]
    return row.iloc[0]['STATE'] if not row.empty else no_update

# Updating the County dropdown options based on the selected state or map click
@app.callback(
    [Output('county-dd', 'options'), Output('county-dd', 'value')],
    [Input('state-dd', 'value'), Input('map', 'clickData')],
    State('county-dd', 'value')
)
def update_county_list(state, click_data, current_county):
    df_state = shape_wgs84[shape_wgs84['STATE'] == state]
    counties = sorted(df_state[NAME_COL].dropna().unique().tolist())
    options = [{'label': c, 'value': c} for c in counties]
    # If the map was clicked, select the specific county from that click
    if ctx.triggered_id == 'map' and click_data:
        geoid = click_data['points'][0].get('location', '')
        row = shape_wgs84[shape_wgs84['GEOID'] == geoid]
        if not row.empty and row.iloc[0][NAME_COL] in counties:
            return options, row.iloc[0][NAME_COL]
    # Default to current selection or the first county in the list
    val = current_county if current_county in counties else (counties[0] if counties else no_update)
    return options, val

# Updating the horizontal bar chart based on the selected county
@app.callback(
    [Output('bar-chart', 'figure'), Output('click-info', 'children')],
    [Input('county-dd', 'value'), Input('state-dd', 'value')]
)
def update_bar_chart(county, state):
    if not county or not state: return go.Figure(), ""
    row = shape_wgs84[(shape_wgs84[NAME_COL] == county) & (shape_wgs84['STATE'] == state)]
    if row.empty: return go.Figure(), ""
    r = row.iloc[0]
    # Preparing data for horizontal bar chart
    races = ['white', 'black', 'asian', 'native', 'other']
    fig = go.Figure(go.Bar(
        x=[r[f'pop_{race}'] for race in races],
        y=[race.capitalize() for race in races],
        orientation='h',
```



R code:

Here we performed all the tasks previously done in Python using R within RStudio, leveraging libraries such as ggplot2 for the charts, sf for spatial data, and Shiny to build the interactive dashboard

1.Preliminaries

```
install.packages("gridExtra")
library(ipumsr)
library(tidyverse)

mykey <- readLines("C:/Users/chaim/Mag2_S1/ipums_key.txt")
set_ipums_api_key(mykey, save = TRUE,overwrite = TRUE)

nhgis_ext <- define_extract_agg(
  collection="nhgis",
  description = "County shapefiles (2010)",
  shapefiles = "us_county_2010_tl2010"
)
nhgis_ext_submitted <- submit_extract(nhgis_ext)
nhgis_ext_complete <- wait_for_extract(nhgis_ext_submitted)
filepath <- download_extract(nhgis_ext_submitted,
                             download_dir = "C:/Users/chaim/Mag2_S1/")
```

```r
nhgis_ext2 <- define_extract_nhgis(
  description = "CW8: Persons by Sex by Race at county level",
  time_series_tables = tst_spec(
    "CW8",
    geog_levels = c("county"),
    years = c("2010")
  )
)
nhgis_ext2_submitted <- submit_extract(nhgis_ext2)
nhgis_ext2_complete <- wait_for_extract(nhgis_ext2_submitted)
filepath2 <- download_extract(nhgis_ext2_submitted,
                              download_dir = "C:/Users/chaim/Mag2_S1/")
```

```r
library(sf)
map_dir <- "C:/Users/chaim/Mag2_S1/"

shape_us <- read_ipums_sf(filepath)

#Reading path
data_stats <- read_nhgis(filepath2)

# filtering for us contingus
shape_us <- shape_us %>%
  mutate(STATEFP10 = as.integer(STATEFP10)) %>%
  filter(STATEFP10 < 57) %>%
  filter(!(STATEFP10 %in% c(2, 15)))

# Merging both files
# We add data population (data_stats) to the shape_us map
shape_us <- shape_us %>%
  left_join(data_stats, by = "GISJOIN")
```

```r
df_counts <- shape_us %>%
  as_tibble() %>%
  mutate(
    # Summing Male (A-F) + Women (G-L) for each category
    pop_white  = CW8AA2010 + CW8AG2010,
    pop_black  = CW8AB2010 + CW8AH2010,
    pop_native = CW8AC2010 + CW8AI2010,
    pop_asian  = CW8AD2010 + CW8AJ2010,
    pop_other  = CW8AE2010 + CW8AK2010 + CW8AF2010 + CW8AL2010,
    total_pop  = pop_white + pop_black + pop_native + pop_asian + pop_other
  ) %>%
  select(GISJOIN, starts_with("pop_"), total_pop)

shape_us_final <- shape_us %>%
  left_join(df_counts, by = "GISJOIN")
```

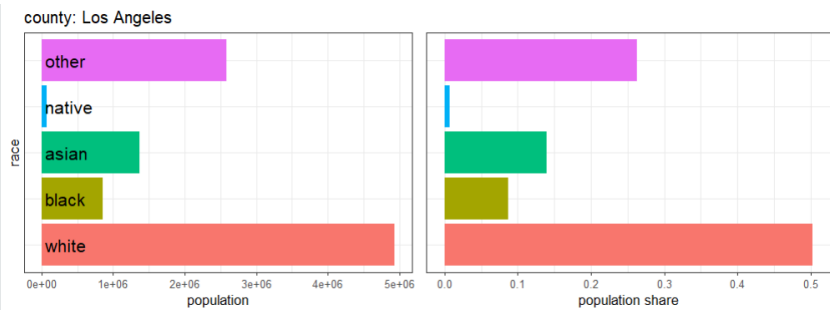
```r
# Selecting only necessary columns
shape_us_final <- shape_us_final %>%
  select(
    GISJOIN, STATEFP10, COUNTYFP10, COUNTYNS10, GEOID10,
    NAME10, NAMELSAD10, LSAD10, CLASSFP10, MTFCC10,
    CSAFP10, CBSAFP10, METDIVFP10, FUNCSTAT10,
    ALAND10, AWATER10, INTPTLAT10, INTPTLON10,
    Shape_len, Shape_area, geometry,
    GEOGYEAR, pop_white, pop_black, pop_native,
    pop_asian, pop_other, total_pop
  )

dim(shape_us_final)

shape_us_final
```

2.Task 1: County population barplots

```r
## TASK1

la_data <- shape_us_final %>%
  filter(GEOID10 == "06037") %>%
  st_drop_geometry() %>%
  pivot_longer(
    cols = c(pop_white, pop_black, pop_asian, pop_native, pop_other),
    names_to = "race",
    values_to = "population"
  ) %>%
  mutate(
    race = str_remove(race, "pop_"),
    share = population / total_pop,
    race = factor(race, levels = c("white", "black", "asian", "native", "other"))
  )

# downloading grid extra
if(!require(gridExtra)) install.packages("gridExtra")
library(gridExtra)
```

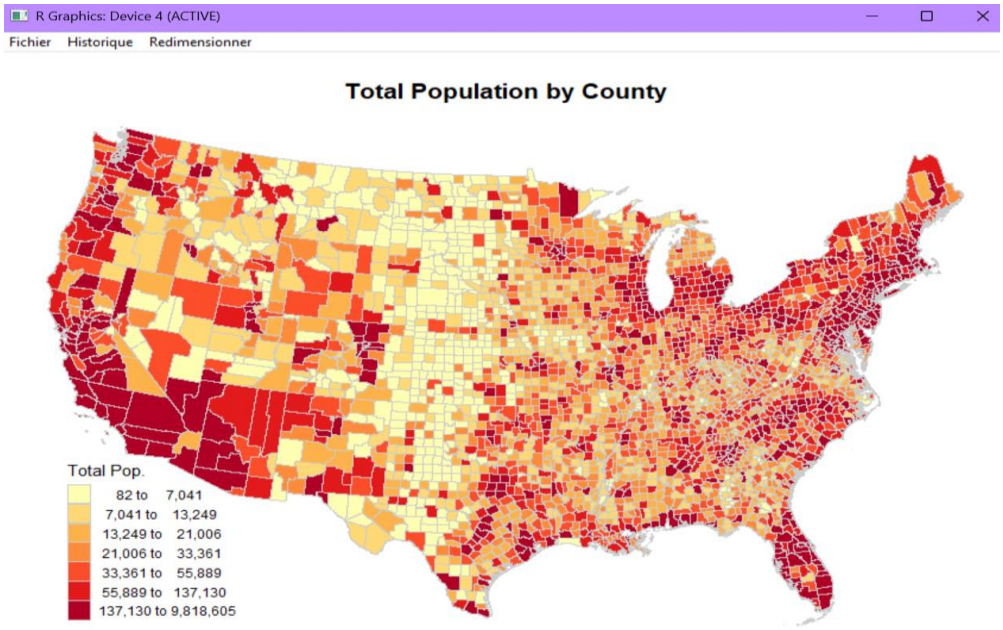
```r
# Defining panel color
colors <- c(
  "white"  = "#f8766d",
  "black"  = "#a3a500",
  "asian"  = "#00bf7d",
  "native" = "#00b0f6",
  "other"  = "#e76bf3"
)

# P1 graph
p1 <- ggplot(la_data, aes(y = race, x = population, fill = race)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = race), x = 50000, hjust = 0, size = 5, color = "black") +
  scale_fill_manual(values = colors) +
  theme_bw() +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank()) +
  labs(title = "county: Los Angeles", x = "population", y = "race")
```

```r
# p2 graph
p2 <- ggplot(la_data, aes(y = race, x = share, fill = race)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = colors) +
  theme_bw() +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank()) +
  labs(title = "", x = "population share", y = NULL)

grid.arrange(p1, p2, ncol = 2)
```



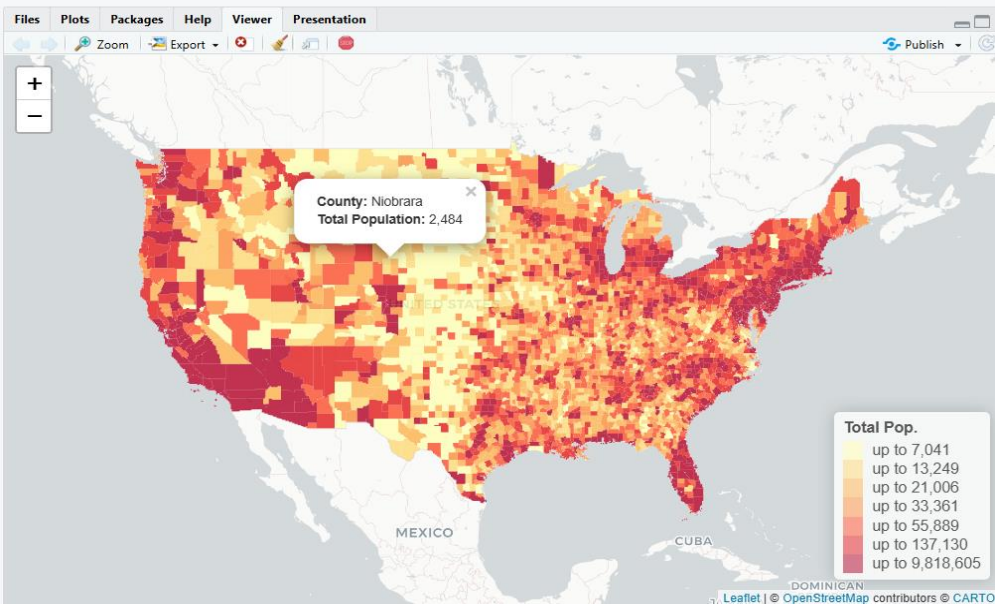
3. Task 2: All counties

```r
#TASK2
library(ggplot2)
library(RColorBrewer)
library(scales)

bks <- quantile(shape_us_final$total_pop,
                probs = seq(0, 1, length.out = 8),
                na.rm = TRUE)

# creating label X and Y
labels_leg <- paste0(format(round(bks[-8]), big.mark=","),
                     " to ",
                     format(round(bks[-1]), big.mark=","))

shape_us_final$pop_cat <- cut(shape_us_final$total_pop,
                              breaks = bks,
                              labels = labels_leg,
                              include.lowest = TRUE)

p <- ggplot(data = shape_us_final) +
  geom_sf(aes(fill = pop_cat), color = "grey80", linewidth = 0.1) +
  scale_fill_manual(
    values = brewer.pal(7, "YlOrRd"),
    name = "Total Pop.",
    na.value = "grey50"
  ) +
  labs(title = "Total Population by County") +
  theme_void() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 16),
    legend.position = c(0.15, 0.2),
    legend.key.size = unit(0.5, "cm")
  )

windows()
print(p)
```



4. Task 3: leaflet

```
# TASK 3 - Leaflet
library(leaflet)
library(RColorBrewer)
library(sf)
library(htmlwidgets)

counties_WGS84 <- sf::st_transform(shape_us_final, 4326)

pal_fun <- colorQuantile("YlOrRd", NULL, n = 7)
bks <- quantile(counties_WGS84$total_pop, probs = seq(0, 1, length.out = 8), na.rm = TRUE)

p_popup <- paste0(
  "<strong>County: </strong>", counties_WGS84$NAME10, "<br>",
  "<strong>Total Population: </strong>", format(counties_WGS84$total_pop, big.mark = ",")
)

my_map <- leaflet(counties_WGS84) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addPolygons(
    stroke = FALSE,
    fillColor = ~pal_fun(total_pop),
    fillOpacity = 0.8,
    smoothFactor = 0.5,
    popup = p_popup
  ) %>%
  addLegend(
    "bottomright",
    colors = brewer.pal(7, "YlOrRd"),
    labels = paste0("up to ", format(bks[-1], big.mark = ",", digits = 2)),
    title = "Total Pop."
  )

saveWidget(my_map, "map_counties.html", selfcontained = TRUE)
browseURL("map_counties.html")
```



5.Task 4: Shiny

```
library(shiny)
library(leaflet)
library(dplyr)
library(ggplot2)
library(sf)
library(scales)
library(gridExtra)

fips_to_state <- c(
  "01"="Alabama","04"="Arizona","05"="Arkansas","06"="California",
  "08"="Colorado","09"="Connecticut","10"="Delaware","11"="DC",
  "12"="Florida","13"="Georgia","16"="Idaho","17"="Illinois",
  "18"="Indiana","19"="Iowa","20"="Kansas","21"="Kentucky",
  "22"="Louisiana","23"="Maine","24"="Maryland","25"="Massachusetts",
  "26"="Michigan","27"="Minnesota","28"="Mississippi","29"="Missouri",
  "30"="Montana","31"="Nebraska","32"="Nevada","33"="New Hampshire",
  "34"="New Jersey","35"="New Mexico","36"="New York","37"="North Carolina",
  "38"="North Dakota","39"="Ohio","40"="Oklahoma","41"="Oregon",
  "42"="Pennsylvania","44"="Rhode Island","45"="South Carolina",
  "46"="South Dakota","47"="Tennessee","48"="Texas","49"="Utah",
  "50"="Vermont","51"="Virginia","53"="Washington","54"="West Virginia",
  "55"="Wisconsin","56"="Wyoming"
)

shape_wgs84 <- shape_us_final %>%
  st_transform(crs = 4326) %>%
  rename(GEOID = GEOID10) %>%
  mutate(
    STATE = fips_to_state[sprintf("%02d", as.integer(STATEFP10))]
  )

shape_wgs84$geometry <- st_simplify(shape_wgs84$geometry, dTolerance = 0.01)

NAME_COL <- "NAME10"
states   <- sort(unique(na.omit(shape_wgs84$STATE)))

RACE_COLORS <- c(
  white  = "#F08080", black  = "#808000",
  asian  = "#2E8B57", native = "#1E90FF", other  = "#DA70D6"
)
```

```
ui <- fluidPage(
  tags$head(tags$style(HTML("
    body { font-family: Arial; }
    .header { background:#2c3e50; color:white; text-align:center;
              padding:10px; font-size:18px; margin-bottom:0; }
    .sidebar { width:200px; padding:10px; border-right:1px solid #eee;
               background:#f9f9f9; min-height:600px; }
    .main-panel { flex:1; padding:5px; }
    .app-body { display:flex; border:1px solid #ddd; max-width:1000px; margin:auto; }
  ))),

  div(class = "header", "US County Population (2010)"),

  div(class = "app-body",
      div(class = "sidebar",
          tags$b("State:"),
          selectInput("state_dd", label = NULL, choices = states, selected = "California"),
          tags$br(),
          tags$b("County:"),
          selectInput("county_dd", label = NULL, choices = NULL),
          tags$br(),
          uiOutput("click_info")
      ),
      div(class = "main-panel",
          leafletOutput("map", height = "350px"),
          plotOutput("bar_chart", height = "250px")
      )
  )
)
```

```r
# SERVER
server <- function(input, output, session) {

  pal <- colorNumeric("YlOrRd", domain = shape_wgs84$total_pop)

  output$map <- renderLeaflet({
    leaflet(shape_wgs84) %>%
      addProviderTiles("CartoDB.Positron") %>%
      setView(lng = -97, lat = 38, zoom = 4) %>%
      addPolygons(
        fillColor   = ~pal(total_pop),
        fillOpacity = 0.7,
        color       = "#FFFFFF",
        weight      = 0.5,
        layerId     = ~GEOID,
        label       = ~NAME10
      ) %>%
      addLegend("bottomright", pal = pal, values = ~total_pop, title = "Population")
  })

  observeEvent(input$map_shape_click, {
    geoid <- input$map_shape_click$id
    row   <- shape_wgs84[shape_wgs84$GEOID == geoid, ]
    if (nrow(row) == 0) return()

    clicked_state  <- row$STATE[1]
    clicked_county <- row[[NAME_COL]][1]

    updateSelectInput(session, "state_dd", selected = clicked_state)

      tags$b(r$NAME10),
      tags$p(paste0("Pop: ", format(as.integer(r$total_pop), big.mark = ",")))
    )
  })
}

shinyApp(ui, server)
```

```r
  counties_in_state <- shape_wgs84 %>%
    filter(STATE == clicked_state) %>%
    pull(NAME10) %>% na.omit() %>% sort() %>% unique()

  updateSelectInput(session, "county_dd",
                    choices = counties_in_state, selected = clicked_county)
})

observeEvent(input$state_dd, {
  counties <- shape_wgs84 %>%
    filter(STATE == input$state_dd) %>%
    pull(NAME10) %>% na.omit() %>% sort() %>% unique()

  updateSelectInput(session, "county_dd",
                    choices = counties, selected = counties[1])
})

selected_row <- reactive({
  req(input$county_dd, input$state_dd)
  shape_wgs84 %>%
    filter(NAME10 == input$county_dd, STATE == input$state_dd)
})

output$bar_chart <- renderPlot({
  r <- selected_row()
  req(nrow(r) > 0)
  r <- r[1, ]

  races <- c("white", "black", "asian", "native", "other")
  pops  <- sapply(races, function(rc) as.numeric(r[[paste0("pop_", rc)]]))
  total <- sum(pops)

  df <- data.frame(
    race  = factor(tools::toTitleCase(races), levels = rev(tools::toTitleCase(races))),
    pop   = pops,
    share = ifelse(total > 0, pops / total, 0),
    color = RACE_COLORS[races]
  )

  p1 <- ggplot(df, aes(x = pop, y = race, fill = race)) +
    geom_col() +
    scale_fill_manual(values = setNames(df$color, df$race)) +
    scale_x_continuous(labels = comma) +
    labs(x = "Population", y = NULL, title = "Count") +
    theme_minimal() + theme(legend.position = "none")

  p2 <- ggplot(df, aes(x = share, y = race, fill = race)) +
    geom_col() +
    scale_fill_manual(values = setNames(df$color, df$race)) +
    scale_x_continuous(labels = percent) +
    labs(x = "Share", y = NULL, title = "Proportion") +
    theme_minimal() + theme(legend.position = "none", axis.text.y = element_blank())

  grid.arrange(p1, p2, ncol = 2)
})

output$click_info <- renderUI({
  r <- selected_row()
  req(nrow(r) > 0)
  r <- r[1, ]
  tagList(
    tags$b(r$NAME10),
```